Chapitre 1: Introduction à la programmation Python

Python : Caractéristiques et particularités

Python: Langage du débutant

Très simple et intuitif à apprendre



Très simple et intuitif à apprendre



Des bibliothèques puissantes



Gratuit et open source



Une communauté et une documentation remarquables

Python : Très simple et intuitif à apprendre – Code court et concis

- En Java, C et C++, la syntaxe utilise beaucoup d'accolades « { } » et de points-virgules « ; ».
- En Java, il faut également préciser de nombreux éléments, comme « public ».
- À l'inverse, Python privilégie une syntaxe simple, claire et concise.

Exemple: Afficher "Hello World"

Java

```
public class HelloWorld {
    public static void main( String[] args ) {
        System.out.println( "Hello World!" );
        System.exit( 0 );
    }
}
```

- En Python, un script est un fichier contenant une suite d'instructions que l'on peut exécuter directement avec l'interpréteur Python (`Python programme.py`).
- Sa syntaxe concise permet d'écrire rapidement des programmes

C

```
#include <stdio.h>

int main(int argc, char **argv) {
    printf("Hello World");
    return 0;
}
```

C++

```
#include <iostream>
using namespace std;

int main(int argc, char** argv) {
    cout << "Hello, World!";
    return 0;
}
```

Python 2

```
print "Hello world"
```

Python 3

```
print("Hello world")
```

Python est un langage de programmation interprété

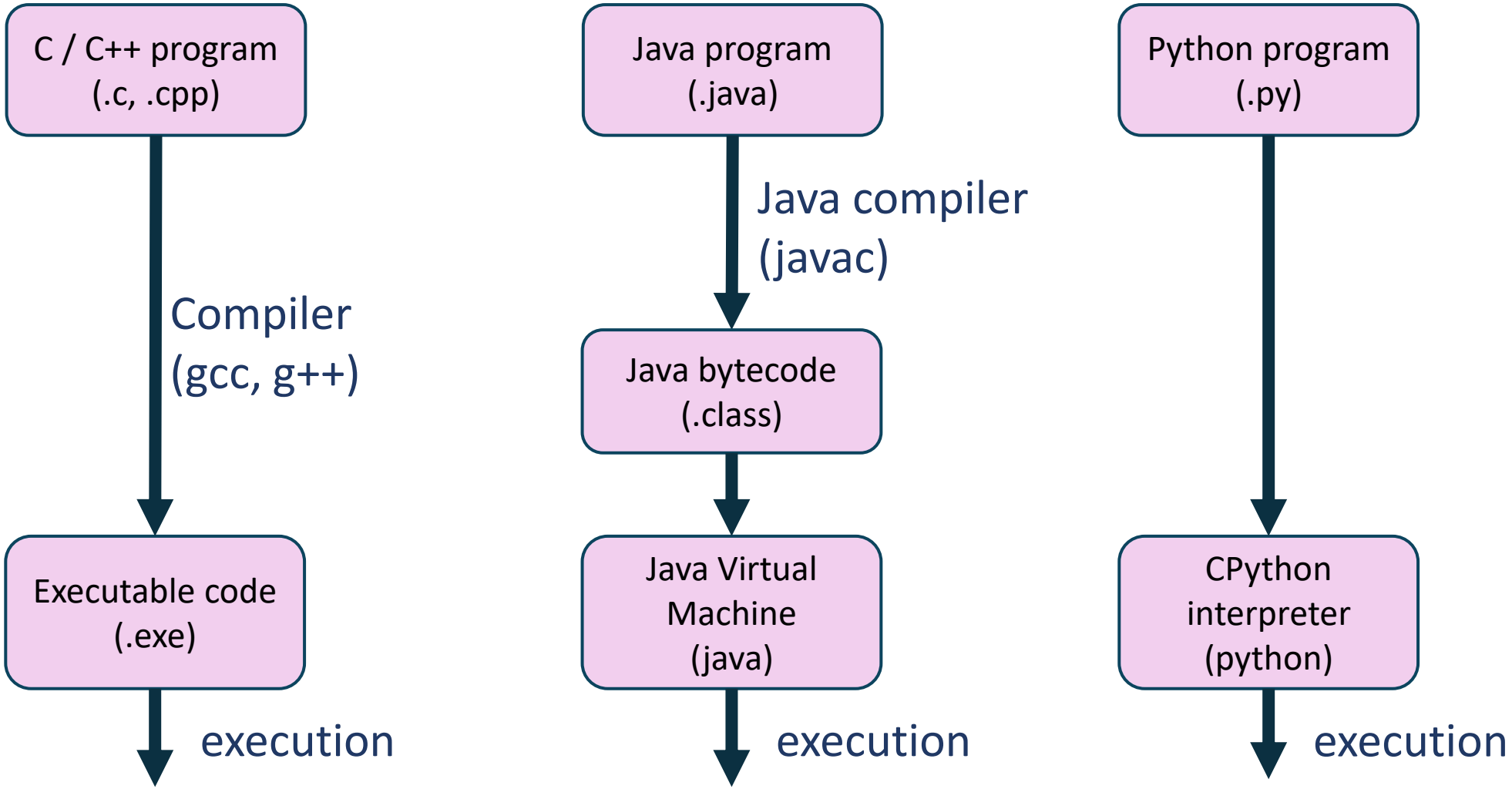


python™

• Certains langages (par ex. C/C++) sont compilés : nous écrivons du code, à partir duquel nous obtenons un programme exécutable par compilation.

• En revanche, Python est interprété : un programme appelé interpréteur exécute directement notre code, ligne par ligne.

Interpréteur vs compilateur



Python est à typage dynamique,

- à typage dynamique : dans de nombreux langages, lorsque vous déclarez une variable, vous devez spécifier son type (par ex. int, double, Boolean, string).

Python est à typage dynamique

•Le type actuel d'une valeur peut être déterminé à l'aide de la fonction type(), qui retourne un objet représentant son type.

•En Python, une variable peut contenir des valeurs de types différents au cours de l'exécution du programme.

•Dans des langages comme C, C++ et Java, les variables sont déclarées avec un type donné, par exemple :

```
int x = 42;
```

Les différentes valeurs affectées à cette variable doivent ensuite respecter ce type.

Python shell

```
> x = 1
> type(x)
| <class 'int'>
> x = 'Hello'
> type(x)
| <class 'str'>
> type(42)
| <class 'int'>
> type(type(42))
| <class 'type'>
```



Python et les performances

- **Langage interprété → exécuté ligne par ligne, ce qui le rend plus lent que les langages compilés.**

- **Temps d'exécution → le typage dynamique et l'interprétation ajoutent une surcharge qui affecte la vitesse**

C ≈ C++ > Java >> Python

- **C en interne**→ L'implémentation standard de Python (**CPython**) est écrite en C.

- **Hautes performances grâce aux bibliothèques → les tâches lourdes (NumPy, TensorFlow, Pandas) reposent sur du code C/C++ optimisé.**

Bibliothèques Python populaires

Library	Usage
numpy, scipy	Scientific & technical computing
pandas	Data manipulation & aggregation
mlpy, scikit-learn	Machine learning
theano, tensorflow, keras	Deep learning
statsmodels	Statistical analysis
nltk, gensim	Text processing
networkx	Network analysis & visualization
bokeh, matplotlib, seaborn, plotly	Visualization
beautifulsoup, scrapy	Web scraping

Caractéristiques avancées de Python

Python prend en charge plusieurs paradigmes de programmation:

- **Programmation procédurale** → le programme est organisé comme une suite d'instructions et de procédures/fonctions qui réalisent des traitements.

- **Programmation orientée objet (POO)** → le programme est organisé autour d'objets qui regroupent des données et les opérations qui peuvent être effectuées sur ces données.

- **Programmation fonctionnelle** → le programme s'appuie sur des fonctions comme éléments fondamentaux, en privilégiant notamment la transformation et la composition des données.

Révision et évaluation des acquis

Questions:

A/

X = [1, 3, 5]

Y= X

X[0] = 2

Print (Y)

C/

- `x = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`

`x[0]? x[1]? x[-1]?  x[-2]?`

`x[1:5] ?`

`x[-3:] ?`

B/

```
> x = 3
> y = 5
> x = 2
> print(x * y)
```

D/

```
a = list(range(10))
b = a
b[0] = 100
print(a)

a = list(range(10))
b = a[:]
b[0] = 100
print(a)
```

Questions

E/

a_tuple = (0, 1, 2, 3, 4)

a_tuple[2:4]

a_tuple[2] = 5

F/

```
grades = { "Joel" : 80, "Tim" : 95 }
joels_grade = grades.get("Joel", 0)?

kates_grade = grades.get("Kate", 0)?
```

grades["Joel"] ?

grades["Kate"] ?

Questions

```
def poly(z, x, y):
    return z * x + y
poly(3, "10", '3`)?
```

```
def subtract(a=0, b=0):
    return a - b
subtract() ?
subtract(10, 5)  ?
subtract(0, 5)  ?
subtract(b = 5)  ?
subtract(b = 5, a = 0) ?
```

```
def greet(name, age=18):
    print(f"Hello {name}, you are {age}")

greet("Alice")    ?
greet("Bob", 30)  ?
```

`f(6, z=2)`?

```
def f(x, y=3, z=7):
    return x + y + z
```

a) `10`

b) `11`

c) `16`

d) `TypeError: f() missing 1 required positional argument: 'y'`

Questions

```
def demo(a, b, *args, **kwargs):
    print(args, kwargs)

demo(1, 2, 3, 4, x=10, y=20)
```

Question:

```
def add(a, b, c=0):
  return a + b + c

def logger (func, *args, **kwargs):
  print(f"Calling function: {func.__name__}")
  print("Positional args:", args)
  print("Keyword args:", kwargs)
  result = func(*args, **kwargs)
  print("Result:", result)
  print("-" * 30)
  return result
```

```
# Utilisation du logger avec add
logger(add, 2, 3)          # Appelle add(2, 3)
logger(add, 1, 2, 3)       # Appelle add(1, 2, 3)
logger(add, 10, 20, c=30)  # Appelle add(10, 20, c=30)
```

Question:

```
def greet(name, greeting="Hello"):
  return f"{greeting}, {name}!"

def logger (func, *args, **kwargs):
  print(f"Calling function: {func.__name__}")
  print("Positional args:", args)
  print("Keyword args:", kwargs)
  result = func(*args, **kwargs)
  print("Result:", result)
  print("-" * 30)
  return result
```

```
# Utilisation du logger avec greet
logger(greet, "Alice")                  # Appelle greet("Alice")
logger(greet, "Bob", "Bonjour")         # Appelle greet("Bob", "Bonjour")
logger(greet, name="Charlie", greeting="Hi") # Appelle greet(name="Charlie", greeting="Hi")
```

Questions

```
import math

def dist(x0, y0, x1, y1):
  return math.sqrt((x1 - x0) ** 2 + (y1 - y0) ** 2)

p = 3, 7
q = 7, 4



dist(p, q)???

dist(*p, *q) ???
```

Question – Quel est le nombre de paramètres de la fonction `f` ?

```
a = [1, 2, 3]
b = [4, 5]
c = (6, 7, 8)
d = (9, 10)
f(*a, b, c, *d)
```

a) 4
b) 5
c) 6
d) 7
e) 8
f) 9
g) 10

Donner la sortie de ce programme.

```
>x = 42
>def f():
      print(x)
      x = 7
>f()
```

Question – Quelle valeur est affichée ?

```
x = 1
def f(a):
    global x
    x = x + 1
    return a + x
print(f(2) + f(4))
```

a) 6
b) 7
c) 8
d) 9
e) 10
f) 11
g) 12
h) Don’t know

```
> def f():
      g()
      def g():
          print('Howdy')
```

- def double(x):
  print(x * 2)

  DD = double
  DD(2)

zip

- zip prend plusieurs itérables et les combine élément par élément en tuples, qui peuvent ensuite être traités par des fonctions comme map, filter ou reduce.

```
list(zip(['a', 'b', 'c'], [1, 2, 3], ['A', 'B', 'C']))
Out: [('a', 1, 'A'), ('b', 2, 'B'), ('c', 3, 'C')]

names = ['James', 'Tom', 'Mary']
grades = [100, 90, 95]
list(zip(names, grades))
Out: [('James', 100), ('Tom', 90), ('Mary', 95)]
```

Argument unpacking

```
gradeBook = [['James', 100],
                                    ['Tom', 90],
                                    ['Mary', 95]]
[names, grades]=zip(*gradeBook)

print(names)
Out: ('James', 'Tom', 'Mary')
print(grades)
Out: (100, 90, 95)
```

Programmation fonctionnelle en Python

List comprehension

- Exemple: Créer une liste des carrés des nombres de 0 à 4.

- Solution 1:

```
squares = []
for x in range(5):
        squares.append(x*x)
print(squares)
Out: [0, 1, 4, 9, 16]
```

- Solution 2: (Une manière plus compacte avec les list comprehension)

```
squares = [x * x for x in range(5)]
print(squares)
Out: [0, 1, 4, 9, 16]
```

List comprehension - 2

- Peut également être utilisée pour filtrer une liste.

```
even_numbers = [x for x in range(5) if x % 2 == 0]
print(even_numbers)
Out: [0, 2, 4]

even_numbers = []
for x in range(5):
        if x % 2 == 0:
        even_numbers.append(x)
print(even_numbers)
Out: [0, 2, 4]
```

Lambda functions

lambda argument_list: expression

- Une **fonction lambda** est une manière de créer de petites fonctions anonymes, c'est-à-dire des fonctions sans nom.
  Ces fonctions sont temporaires (*throw-away functions*), c'est-à-dire qu'elles sont utilisées uniquement à l'endroit où elles sont définies.

```
x = lambda a : a + 10
print(x(5))
```

- La puissance des fonctions **lambda** apparaît surtout lorsqu'on les utilise comme fonctions anonymes à l'intérieur d'une autre fonction (voir **map** ci-après).

Exemple

```
def apply (f, n):
  return f(n)


print ( apply ( lambda x : x + 1, 41) )    ?
print ( apply ( lambda x : x ** 2, 9) )    ?
```

Fonctions all et any

Tous les/au moins un élément d'une séquence vaut True Les fonctions all et any renvoient un booléen

```
data = [2 , -6, 0, 8]
test = lambda e : e % 2 == 0 and e >= 0
filtered = [ test (e) for e in data ]
Filtered # [True, False, True, True]
print ( all ( filtered )) # False
print ( any ( filtered )) #True
```

Functools: **map**, reduce, filter

- Application d'une fonction à tous les éléments d'une séquence ***map*** calcule une nouvelle séquence et la renvoie
- Résultat renvoyé sous la forme d'un générateur
- Utilisation de la fonction list pour le convertir

map(function_to_apply, list_of_inputs)

```
items = [1, 2, 3, 4, 5]
squared = []
for i in items:
    squared.append(i**2)

#using map allows us to implement in simpler way
items = [1, 2, 3, 4, 5]
squared = list(map(lambda x: x**2, items))
```

Question

list1 = [2, 3, 4]

list2 = [5, 6, 7]

Écris un programme Python qui multiplie les éléments de deux listes **élément par élément** (utilise l'outil map)

Functools: **map**, reduce, filter

Cas général : plusieurs itérables

items1 = [1, 2, 3]
items2 = [4, 5, 6]
items3 = [7, 8, 9]

f = lambda a, b, c: a + b + c
iterables = [items1, items2, items3]

- result = list(map(f, *iterables))
- print(result) # [12, 15, 18]

Question

```
items = [1, 2, 3, 4, 5]
result = list(map(lambda x, y: x**2 + y**2, items, items))
print(result)
```

- À partir de la liste suivante de valeurs en Celsius, générez les valeurs correspondantes en Fahrenheit en utilisant map et une fonction lambda.

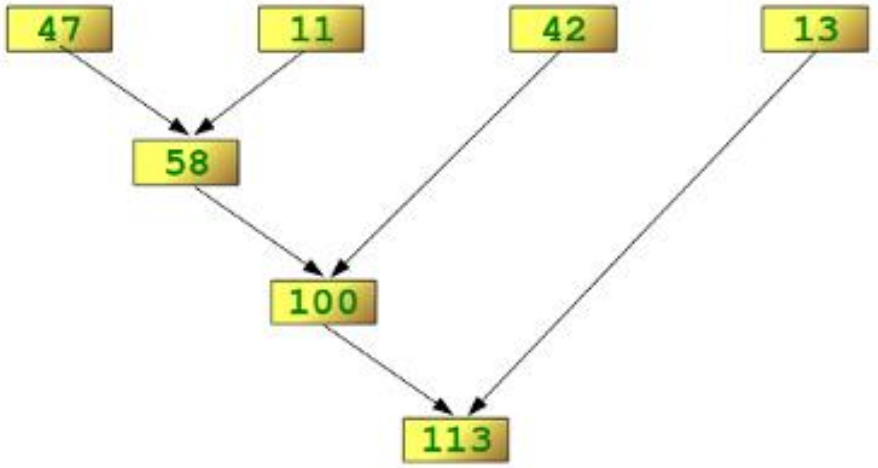
Celsius = [39.2, 36.5, 37.3, 37.8]

F = ((9/5)*C) + 32.0

Functools: map, **reduce**, filter

- La fonction reduce(func, seq) applique successivement la fonction func() aux éléments de la séquence seq et renvoie une seule valeur

```
reduce(lambda x,y: x+y, [47,11,42,13])
```



#output : 113

Question 1

Déterminer le maximum d'une liste de valeurs numériques en utilisant **reduce** :

[47,11,42,102,13])

#output: 102

```
f = lambda a,b: a if (a > b) else b


from functools import reduce

reduce(f, [47,11,42,102,13])
```

Question 2

Calculer la somme des nombres de 1 to 100:

Question 3:

- À l'aide de **reduce** et d'une fonction *gcd* basée sur l'algorithme d'Euclide, calculez le PGCD de la liste [48, 64, 16]

Functools: map, reduce, **filter**

- Garde les éléments d'une séquence satisfaisant une condition ***filter*** calcule une nouvelle séquence et la renvoie Résultat renvoyé sous la forme d'un générateur
- Utilisation de la fonction list pour le convertir

```
number_list = range(-5, 5)
less_than_zero = list(filter(lambda x: x < 0, number_list))
print(less_than_zero)
# Output: [-5, -4, -3, -2, -1]
```

Question – What is `list(zip(*zip(*L)))` ?

```
L = [[1, 2, 3], [4, 5], [6, 7, 8]]
```

a) `[(([1, 2, 3],),), (([4, 5],),), (([6, 7, 8],),)]`

b) `[((1, 4, 6),), ((2, 5, 7),)]`

c) `[(1, 2), (4, 5), (6, 7)]`

d) `[(1, 2, 3), (4, 5), (6, 7, 8)]`

e) `[([1, 2, 3], [4, 5], [6, 7, 8])]`

f) Don’t know

Question

Considérer la fonction suivante:

```
> def triple_sum(x, y, z):
      return x + y + z
```

Déterminer la sortie de chaque instruction

```
> list(map(triple_sum, [1,2,3], [4,5,6], [7,8,9]))?
> list(map(triple_sum, *zip(*[(1,4,7), (2,5,8), (3,6,9)])))?
```

Exercice:

```
data = [
 [2, 1, 2],
 [4, 1, 2],
 [6, 4, 9]
    ]
```

- Transposer la matrice data
- Normaliser chaque caractéristique pour que toutes les valeurs soient comprises entre 0 et 1, en utilisant la formule Min-Max :
  - $x_{norm} = \frac{x - \min f}{\max f - \min f}$
  - Si toutes les valeurs d'une caractéristique sont identiques ($\max f = \min f$), remplacer toutes les valeurs par 0.
  - Utiliser map et lambda
- Retourner le tableau normalisé

Wrapper et Decorator

- Ajouter un comportement à une fonction

Dans cet exemple, la Fonction ***decorate*** qui prend une fonction en paramètre ajoute un comportement avant et après l'appel d'une fonction

```
def decorate (f):
  def wrapper () :
    print ("Avant appel ")
    f()
    print ("Après appel ")
  return wrapper
```

```
@decorate
def sayhello () :
  print ("Hello !")
sayhello ()
```

=

sayhello = decorate(sayhello)

Decorator

- Une décoration est ajoutée explicitement à une fonction
  - Se note avec @ suivi du nom de la fonction décorante

- Un sucre syntaxique est une notation simplificatrice
  - sayhello = decorate(sayhello)

Question – Donner la sortie de ce programme.

decorator_quizz.py

```
def double(f):
    def wrapper():
        return 2 * f()
    return wrapper

def add_three(f):
    def wrapper():
        return 3 + f()
    return wrapper

@double
@add_three
def seven():
    return 7

print(seven())
```

- 7
- 10
- 14
- 17
- 20
- Don’t know

Decorator avec arguments

```
def repeat(n):                          # Decorator argument
    def decorator(f):
        def wrapper(*args, **kwargs):
            for i in range(n):
                f(*args, **kwargs)
        return wrapper
    return decorator


@repeat(3)
def greet(name):                        # Original function with parameter
    print(f"Hello, {name}!")
```

Programmation OOP en Python

Classe simple

```
# Classe simple (pages 55-60 du cours)
class Etudiant:
    def __init__(self, nom, prenom):
        self.nom = nom
        self.prenom = prenom
        self.notes = []


    def ajouter_note(self, note):
        self.notes.append(note)


    def moyenne(self):
        return sum(self.notes) / len(self.notes) if self.notes else 0


    def __str__(self):
        return f"{self.prenom} {self.nom} - Moyenne : {self.moyenne():.2f}"


# Utilisation
e1 = Etudiant(« Ahmed", « Ben Ahmed")
e1.ajouter_note(18)
e1.ajouter_note(17.5)
print(e1)
```

Heritage

```
class EtudiantMaster(Etudiant):
    def __init__(self, nom, prenom, stage):
        super().__init__(nom, prenom)
        self.stage = stage

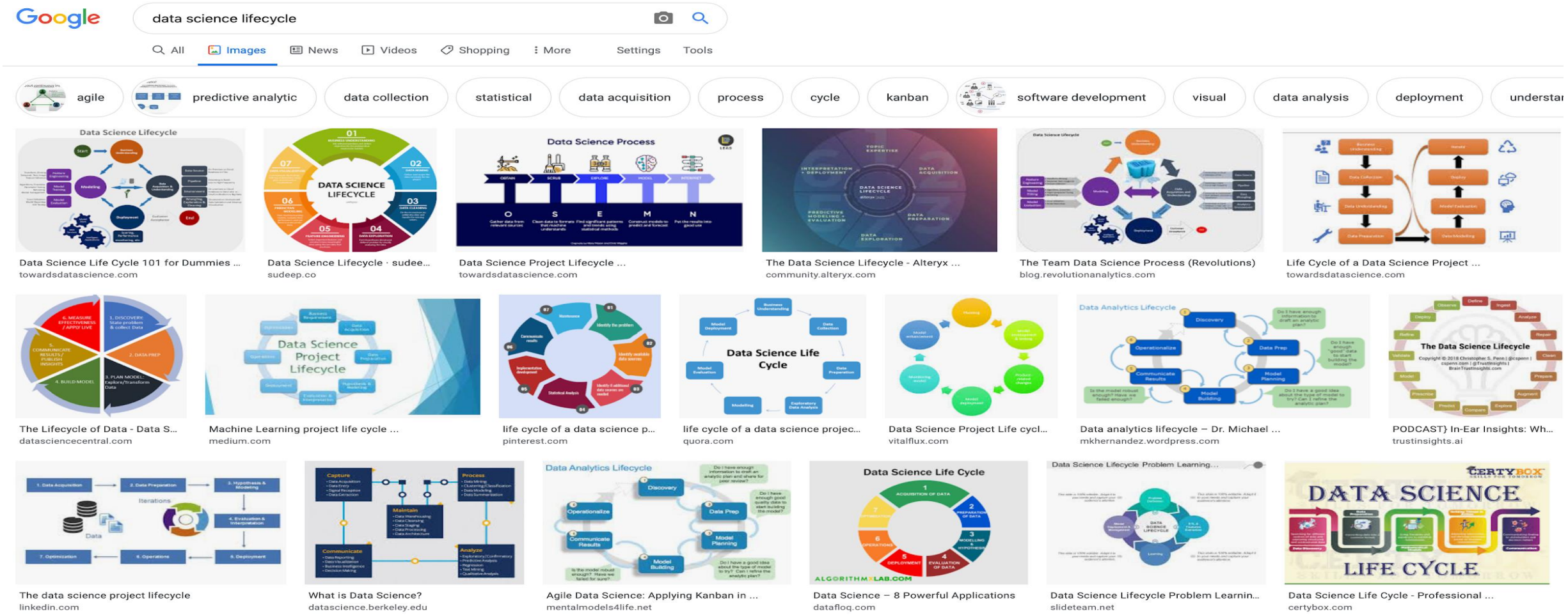

    def __str__(self):
        return super().__str__() + f" - Stage : {self.stage}"


em = EtudiantMaster("Rahali", "Oussama", "Google")
em.ajouter_note(18)
em.ajouter_note(17.5)
print(em)
```

Cycle de vie de la data science

Cycle de vie de la data science

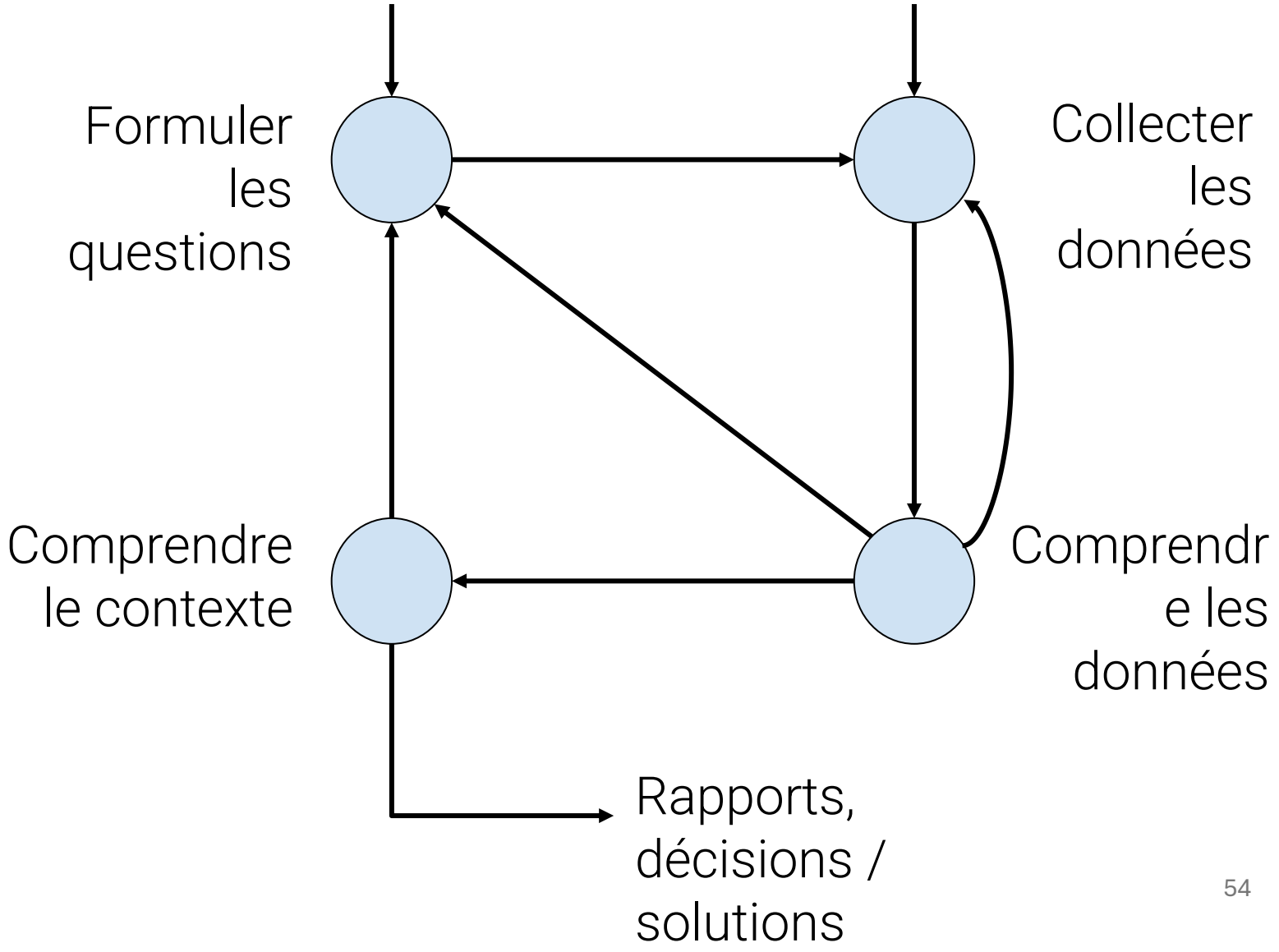
Le cycle de vie d’un projet en data science peut varier selon le contexte professionnel, mais ses principes essentiels demeurent identiques



Cycle de vie de la data science

• Le cycle de vie représente une **vue d'ensemble du workflow** en data science.

• Le processus est **itératif**, impliquant des allers-retours fréquents entre les différentes étapes.



Data Science

From data to decisions

1. Problem & Domain Context

- Industry / Domain knowledge
- Business case & objectives
- Problem formulation
- Stakeholders & requirements
- Privacy, ethics and responsible AI

2. Data Engineering

- Data acquisition (APIs, databases, files, web)
- Data modeling (schemas, data lakes/warehouses)
- Data quality (cleaning, validation, missing data)
- Data management (ETL/ELT, pipelines)
- Scalability & Big Data (distributed systems)

3. Statistical & Machine Learning Modeling

- Exploratory Data Analysis (EDA)
- Statistical inference & hypothesis testing
- Predictive modeling (regression, classification)
- Machine learning (supervised / unsupervised)
- Deep learning (neural networks)
- Model evaluation & validation

4. Communication & Decision

- Data visualization (charts, dashboards, maps)
- Interpretation of results
- Storytelling & presentation
- Business intelligence & reporting
- Decision support & impact

The Power of Python in Data Science

